



TRƯỜNG ĐH KHOA HỌC TỰ NHIÊN - ĐHQG TP HCM
KHOA TOÁN - TIN HỌC
BỘ MÔN ỨNG DỤNG TIN HỌC



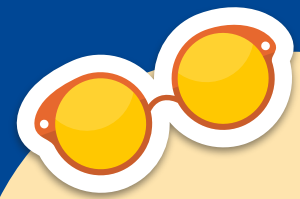
SORT ALGORITHMS

THỰC HÀNH CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT
TUẦN 05+06 - THÁNG 04 NĂM 2026



SORT

Bubble Sort
Selection Sort
Insertion Sort



CONTENTS

01

Selection Sort

Giới thiệu
Bài tập tại lớp

02

Insertion Sort

Giới thiệu
Bài tập tại lớp

03

Exercises

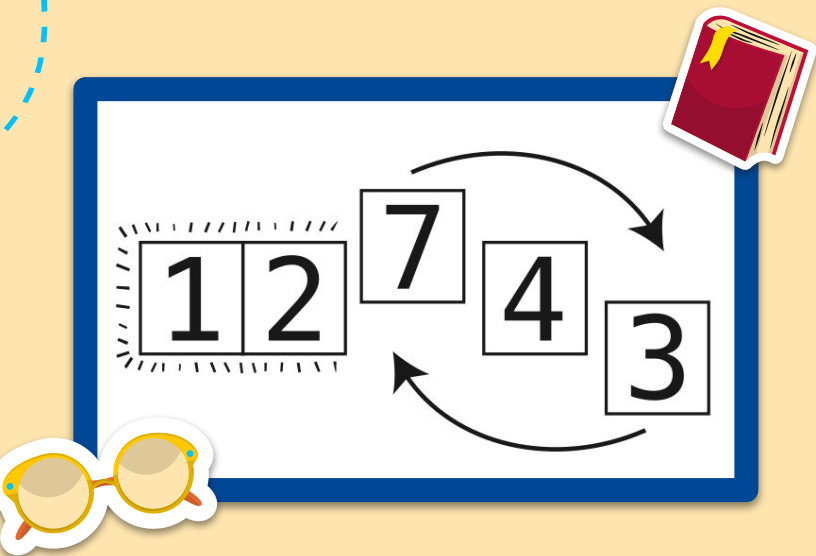
Bài tập tổng hợp

01

Selection Sort



Giới thiệu



Selection sort hoạt động bằng cách tìm kiếm phần tử nhỏ nhất trong mảng và đưa nó vào vị trí đầu tiên. Sau đó, nó tìm phần tử nhỏ nhất tiếp theo và đưa vào vị trí tiếp theo, và tiếp tục quá trình này cho đến khi mảng được sắp xếp hoàn chỉnh.

Mặc dù **selection sort** đơn giản và dễ hiểu, nhưng độ phức tạp của thuật toán là $O(n^2)$. Điều này có nghĩa là thời gian thực hiện sắp xếp tăng theo bình phương số lượng phần tử. Vì vậy, nếu mảng có kích thước lớn thì **selection sort** có thể trở nên chậm và không hiệu quả.

Tuy nhiên, **selection sort** có một ưu điểm là không yêu cầu bộ nhớ phụ hoặc không gian bổ sung. Nó chỉ hoạt động trực tiếp trên mảng ban đầu. Điều này làm cho **selection sort** hữu ích trong một số trường hợp đặc biệt khi không có nhiều bộ nhớ sẵn có.

BÀI TẬP TẠI LỚP

Bài 1: Thực hiện tất cả yêu cầu sau trong một chương trình C:

1. Viết hàm `selectionSort(int arr[], int n)` với chức năng sắp xếp một mảng số nguyên theo thứ tự tăng dần theo thuật toán Selection Sort.
2. Kiểm tra tính hiệu quả của hàm `selectionSort` ở câu 1 bằng một mảng số nguyên kích thước $n = 10$ sinh ngẫu nhiên.

BÀI TẬP TẠI LỚP

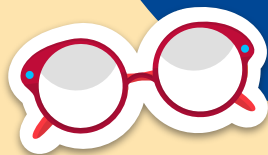
Bài 2: Thực hiện tất cả yêu cầu sau trong một chương trình C:

1. Bổ sung hai biến comps và swaps với chức năng đếm số bước so sánh và số bước hoán đổi

```
selectionSort(int arr[], int n, int*comps, int*swaps)
```

của thuật toán. Kiểm tra hàm trên với mảng số nguyên sinh ngẫu nhiên có độ dài là $n = 10$.

2. Thực hiện hàm trên k lần ($k = 100, 1000, 10000, 100000$) với mỗi lần là một mảng số nguyên được khởi tạo ngẫu nhiên. Với mỗi trường hợp của k, tính số comps trung bình, số swaps trung bình và số step trung bình. **Lập bảng tần số cho swaps.**



Insertion sort



Giới thiệu

Insertion Sort là một thuật toán sắp xếp đơn giản và hiệu quả được sử dụng để sắp xếp danh sách các phần tử. Thuật toán hoạt động bằng cách chia danh sách thành hai phần: một phần đã được sắp xếp và một phần chưa được sắp xếp. Ban đầu, phần đã được sắp xếp chỉ chứa một phần tử đầu tiên của danh sách ban đầu, trong khi phần chưa được sắp xếp chứa các phần tử còn lại.

Thuật toán **Insertion Sort** tiến hành duyệt qua từng phần tử trong phần chưa được sắp xếp và chèn chúng vào đúng vị trí trong phần đã được sắp xếp. Để chèn một phần tử vào phần đã được sắp xếp, ta so sánh nó với các phần tử trong phần đã được sắp xếp từ cuối cùng đến đầu danh sách. Nếu phần tử được chèn nhỏ hơn phần tử hiện tại, ta di chuyển phần tử hiện tại sang phải một vị trí để tạo khoảng trống cho phần tử mới, sau đó chèn phần tử mới vào vị trí này.

Giới thiệu

Quá trình trên được lặp lại cho tất cả các phần tử chưa được sắp xếp cho đến khi danh sách được sắp xếp hoàn toàn. Khi tất cả các phần tử đã được chèn vào phần đã được sắp xếp, ta sẽ có một danh sách đã được sắp xếp.

Đặc điểm của **Insertion Sort** là nó hoạt động tốt với danh sách nhỏ hoặc đã gần sắp xếp, và có thể sử dụng hiệu quả với danh sách có số lượng phần tử nhỏ. Tuy nhiên, với danh sách lớn và chưa sắp xếp, thuật toán này có thể trở nên không hiệu quả so với các thuật toán sắp xếp khác như Quick Sort hoặc Merge Sort.

Độ phức tạp thời gian trung bình của **Insertion Sort** là $O(n^2)$, trong đó n là số lượng phần tử trong danh sách.

BÀI TẬP TẠI LỚP

Bài 3: Hoàn thành hàm `insertionSort` cho phép sắp xếp một mảng số nguyên theo thứ tự tăng dần.

1. In ra màn hình từng bước sắp xếp mảng bằng thuật toán `insertionSort` bên.
2. Thêm biến **comps** (so sánh) và **shifts** (dời) vào hàm `insertionSort` để đếm số bước so sánh và dời phần tử trong quá trình thực hiện `insertionSort`. In ra màn hình giá trị của **comps** và **shifts** sau từng vòng và tổng cả quá trình.

```
void insertionSort(int arr[], int n){  
    int i, key, j;  
    //in  
}
```

```
void main(){  
    int n = 5;  
    int arr[10] = {12, 11, 13, 5, 6};  
  
}
```

VD:

Mang ban dau: 12 11 13 5 6

Buoc 1: 11 12 13 5 6

Buoc 2: 11 12 13 5 6

Buoc 3: 5 11 12 13 6

Buoc 4: 5 6 11 12 13

Mang sau khi sap xep: 5 6 11 12 13

BÀI TẬP TẠI LỚP

Bài 4: Cho mảng số nguyên **a** có kích thước $n = 10$ được sinh ngẫu nhiên.

a. Trong cùng một chương trình, viết hai hàm

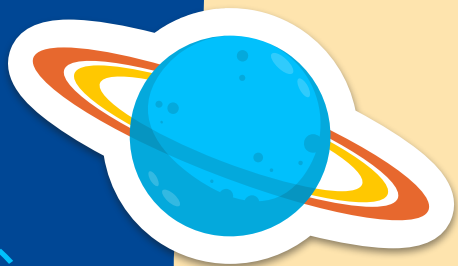
```
selectionSort(int arr[], int n, int*comps, int*swaps)
```

```
void insertionSort(int arr[], int n, int*comps, int*shifts)
```

dùng mảng **a** để tính **số bước tổng trung bình** của `selectionSort` và `insertionSort` sau k lần sinh mảng **a**. ($k = 100, 1000, 10000, 100000$).

Em có nhận xét gì về **số bước tổng trung bình** của hai thuật toán trên?

b. Lập 01 bảng tần số cho `swaps` và `shifts`.



Exercise





Bài 1: Đếm số bước thực hiện của ba thuật toán sắp xếp bubbleSort, selectionSort và insertionSort với mảng

$\{3, 1, 8, 2, 6, 5, 3, 9, 1, 0\}$

Bài 2: Cài đặt thuật toán chỉnh sửa bubbleSort, selectionSort và insertionSort để sắp xếp mảng số nguyên kích thước $n = 10$ sinh ngẫu nhiên theo thứ tự giảm dần. **Trình bày ý tưởng chỉnh sửa dựa trên các thuật toán gốc tương ứng.**





Bài 3: Thực hiện thống kê số bước thực hiện với các mảng số nguyên kích cỡ $n = 10, 20, 50, 100$ sinh ngẫu nhiên của cả 3 thuật toán. Từ đó rút ra bảng so sánh sự giống và khác trong tính hiệu quả của cả 3 thuật toán sắp xếp trên.

